

AFRICAN INSTITUTE FOR MATHEMATICAL SCIENCES

(AIMS RWANDA, KIGALI)

Name: Joachim SINYABE DANBE

Assignment Number: 1

Course: Statistical Machine Learning

Date: December 9, 2025

Video: Kappa concept : number of observations per features in Statistical Machine Learning

Exercise 1 : On The Nine Wheels of SML

Problem Chosen : Prediction of the Remaining Useful Life (RUL) of Turbojet Engines.

We develop a regression system to estimate the number of remaining operational cycles (Continuous values in $\mathbb{R}^+$) before an engine fails, based on sensor readings and operational settings.

Wheel 1 : Problem formulation

We formulate this as a supervised regression problem: given a sequence of multivariate sensors reading for turbofan, we predict a positive target $Y \in \mathbb{R}^+$, the RUL, measured in remaining operation cycles until failure. The input x_i is a time serie of sensor vectors over recent cycles.

So we the mathematically formulation can be write as :

$$Y = f(\mathbf{x}) + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2) \quad (1)$$

where:

- $\mathbf{x} = (x_1, x_2, \dots, x_p)^\top$ is the feature input vetor which are sensors, opperrating, operatin sequence, cycle.
- $f : \mathbb{R}^p \rightarrow \mathbb{R}^+$ the unknown regression function which represent our nore important goal
- ϵ is a pssible random noise

We will try to learn an estimator $\hat{f}$ from training data $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ such that ,

$$\hat{f} = \arg \min_{f \in \mathcal{H}} \mathbb{E}_{(\mathbf{x}, y) \sim P} [(y - f(\mathbf{x}))^2] \quad (2)$$

Where $\mathcal{H}$ is our hypothesis space and P is the unknow data distribution.

Success Metrics: Beyond RMSE.

As we learned The Wrong Question can be ”**What’s the model’s RMSE?**” but the **Right oquestion** is ”does this model reduce maintenance costs while maintaining safety?”.

Why regression and not classification ? Regression is the natural formulation because RUL is are continuous and we don’t only want to know whether and engine ”Will fail soon”, but how soon of cycles. A binary classifier doesn’t take this notion of urgency, whereas a regression model provides a quantitative risk estimate that can drive cost-sensitive dicisions such as when to schedule maintenance or remove an engine from services.

Wheel 2 : Data collection and exploration

We get the date set from NASA(C-MAPSS) with the following informations as show table 1

Characteristic	Value
Training observations	20,631 time-series records
Training engines	100 units (complete run-to-failure)
Test engines	100 units (censored at unknown cycle)
Features per observation	24 (21 sensors + 3 operational settings)
Target variable	RUL (continuous, $\in \mathbb{R}^+$)
Mean RUL	107.8 cycles
Engine lifetime range	128–362 cycles

Table 1: C-MAPSS FD001 Dataset Summary

We defined the **IBM 5 Vs** as :

- **Volume:** $n = 20,631$ observations $\times p = 24$ features.
- **Velocity:** the data from sensor streams are at real-time (per flight cycle).
- **Variety:** Structured multivariate time series. We have continuous sensors + discrete operational settings.
- **Veracity:** We don’t have missing values in the dataset , but it contains measurement noise ϵ and some of the sensors are constant (zero variance) and must be remove.
- **Value:** Correct RUL estimates can save millions in maintenance costs and prevent safety incidents.

The Data Matrix

We write the full dataset as:

$$\mathbf{X} = \begin{bmatrix} x_{1,1} & x_{1,2} & \cdots & x_{1,24} \\ x_{2,1} & x_{2,2} & \cdots & x_{2,24} \\ \vdots & \vdots & \ddots & \vdots \\ x_{20631,1} & x_{20631,2} & \cdots & x_{20631,24} \end{bmatrix} \in \mathbb{R}^{20631 \times 24}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_{20631} \end{bmatrix} \in \mathbb{R}^{20631} \quad (3)$$

Each row $\mathbf{x}_i^\top$ represents a instant of engine state at a given cycle. Each column $\mathbf{x}_{(:,j)}$ is a feature vector across all observations.

Exploratory Data Analysis

The training RUL distribution is heavily right-skewed:

- More observations at early life stages (high RUL values)
 - Fewer observations near failure (low RUL values).
- So our Model can (maybe) struggle to predict imminent failures accurately

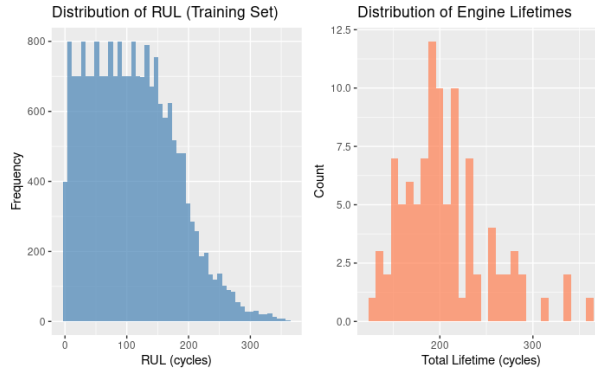


Figure 1: Histogram of RUL distribution showing right skew

The correlation matrix show a strong correlations between sensor_9 and sensor_14 ($r = 0.92$), a moderate RUL correlations between sensor_4, sensor_11, sensor_14 show $|r| > 0.4$ with RUL and a high inter-sensor correlations suggest potential for dimensionality reduction

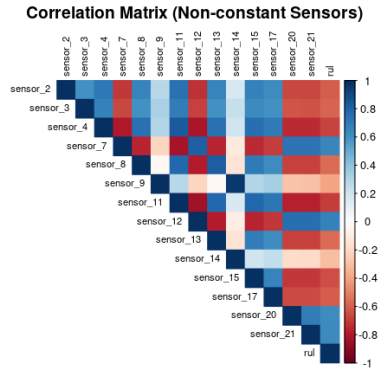


Figure 2: Correlation heatmap of non-constant sensors]

Wheel 3 : Functional space and model choice

We work with three spaces of increasing complexity:
linear regression:

$$\mathcal{H}_{\text{lin}} = \left\{ f(\mathbf{x}) = \beta_0 + \sum_{j=1}^p \beta_j x_j \mid \beta \in \mathbb{R}^{p+1} \right\} \quad (4)$$

This model was fast training, interpretable coefficient, OLS close-form solution but it cannot capture non-linear sensor interactions.

Ridge Regression

$$\mathcal{H}_{\text{ridge}} = \left\{ f(\mathbf{x}) = \beta_0 + \sum_{j=1}^p \beta_j x_j \mid \beta \in \mathbb{R}^{p+1}, \|\beta\|_2^2 \leq C \right\} \quad (5)$$

It handles multicollinearity, shrinks coefficients, prevents overfitting. but the hyperparameter λ requires tuning, still linear .

Random Forest .

$$\mathcal{H}_{\text{RF}} = \left\{ f(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^B T_b(\mathbf{x}) \mid T_b \text{ are decision trees} \right\} \quad (6)$$

,
where each tree T_b is trained on a bootstrap sample with random feature subsampling at each split.

Wheel 4: Learning Principle and Loss

Theoretical Risk Minimization

The ultimate goal is to minimize the **true risk**:

$$f^* = \underset{f \in \mathcal{H}}{\text{argmin}} R(f), \quad R(f) = \int \mathcal{L}(y, f(\mathbf{x})) dP(\mathbf{x}, y) \quad (7)$$

where $\mathcal{L}$ is the loss function and P is the unknown data distribution.

Empirical Risk Minimization (ERM)

The solution is to approximate $R(f)$ using our finite sample $\mathcal{D}_n$:

$$\hat{R}_n(f) = \frac{1}{n} \sum_{i=1}^n \mathcal{L}(y_i, f(\mathbf{x}_i)) \quad (8)$$

Loss Function: Mean Squared Error

For regression, we use:

$$\mathcal{L}(y, \hat{y}) = (y - \hat{y})^2 \quad (9)$$

Justification:

- Convex, differentiable $\Rightarrow$ efficient optimization
- Under Gaussian noise $\epsilon \sim \mathcal{N}(0, \sigma^2)$, MSE minimization = Maximum Likelihood Estimation
- Penalizes large errors quadratically (sensitive to outliers)

Evaluation Metric: RMSE

For interpretability, we report:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (10)$$

RMSE has the same units as RUL (cycles), making it directly interpretable for domain experts.

Wheel 5 : Optimization and computation

We split by *engine ID*, not randomly by s observation, to respect temporal dependencies:

$$\mathcal{D}_{\text{train}} = \{\mathbf{x}_i, y_i \mid \text{engine_id}_i \in \{1, \dots, 80\}\} \quad (n_{\text{train}} = 16,132) \quad (11)$$

$$\mathcal{D}_{\text{val}} = \{\mathbf{x}_i, y_i \mid \text{engine_id}_i \in \{81, \dots, 100\}\} \quad (n_{\text{val}} = 4,499) \quad (12)$$

Rationale: Random splits would leak information (observations from the same engine in both sets), inflating performance estimates.

Algorithm 1: Linear Regression (OLS)

Minimize $\hat{R}_n(\mathbf{w})$ via the normal equations:

$$\hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y} \quad (13)$$

Observation: Modest overfitting gap (6.74 cycles). Linear assumption may be limiting.

Algorithm 2: Ridge Regression

Minimize regularized risk:

$$\hat{\mathbf{w}}_\lambda = \underset{\mathbf{w}}{\text{argmin}} \left\{ \hat{R}_n(\mathbf{w}) + \lambda \|\mathbf{w}\|_2^2 \right\} \quad (14)$$

5-fold cross-validation gives us $\lambda^* = 4.8306$.

But a there is a negligible improvement over OLS. Suggests multicollinearity is not the primary issue.

Algorithm 3: Random Forest

Ensemble of $B = 500$ regression trees with:

- Bootstrap sampling for each tree
- Random feature subset at each split: $m_{\text{try}} = \lfloor \sqrt{p} \rfloor = 4$
- Maximum depth: 30 (to prevent extreme overfitting)

Optimization: Greedy recursive partitioning minimizes within-node MSE.

Random forest has the best validation performance. It is non-linear interactions captured successfully.

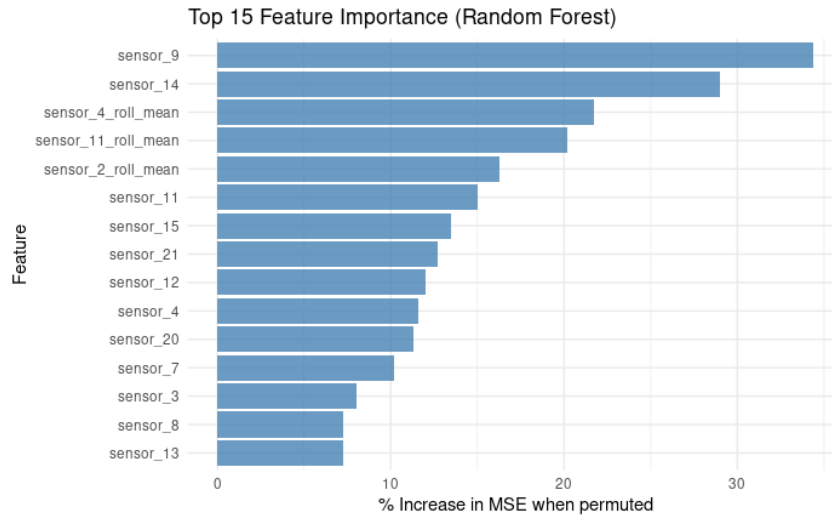


Figure 3: Top 15 feature importances from Random Forest

Wheel 6 : Regularization and refinement

Ridge (L_2 Penalty)

For linear models, we penalize large weights:

$$\Omega(\mathbf{w}) = \|\mathbf{w}\|_2^2 = \sum_{j=1}^p w_j^2 \quad (15)$$

Effect: Shrinks coefficients toward zero, reducing variance at the cost of small bias increase.

Random Forest Regularization

RF employs implicit regularization via:

- **Bootstrap aggregating (bagging):** Variance reduction through ensemble averaging
- **Max depth constraint:** Prevents individual trees from fully memorizing training data
- **Minimum samples per leaf:** Ensures leaf predictions are based on multiple observations

Hyperparameter Selection via Cross-Validation

For Ridge, 5-fold CV estimates generalization error:

$$\hat{R}_{CV}(\lambda) = \frac{1}{5} \sum_{v=1}^5 \hat{R}^{(v)}(f_{-\mathcal{F}_v}; \lambda) \quad (16)$$

where $\mathcal{F}_v$ is the v -th fold and $f_{-\mathcal{F}_v}$ is trained on all data except $\mathcal{F}_v$.

Optimal hyperparameter:

$$\lambda^* = \operatorname{argmin}_{\lambda} \hat{R}_{CV}(\lambda) = 4.8306 \quad (17)$$

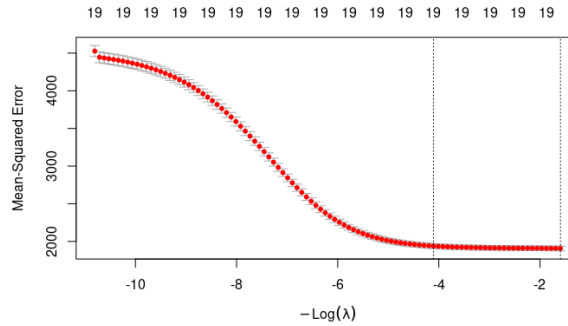


Figure 4

Wheel 7 : Extrinsic Predictive Comparisons

As discussed in Wheel 5, we use an 80-20 engine-level split:

$$\hat{R}_{\text{val}}(f) = \frac{1}{|\mathcal{D}_{\text{val}}|} \sum_{(\mathbf{x}_i, y_i) \in \mathcal{D}_{\text{val}}} \mathcal{L}(y_i, f(\mathbf{x}_i)) \quad (18)$$

Critical principle: As we know, the validation set is *never* used during training or hyperparameter tuning. It provides an unbiased estimate of true risk $R(f)$.

Model	RMSE (Train)	RMSE (Val)	R^2 (Val)	MAE (Val)
Baseline (Mean)	65.69	75.52	0.000	61.32
Linear Regressions	42.06	48.80	0.570	35.81
Ridge Regression	42.10	48.88	0.569	35.82
Random Forest	37.98	47.50	0.593	32.84

Table 2: Model Comparison on Validation Set

Random Forest achieves lowest RMSE (47.50 cycles), representing **37.1% improvement** over naive baseline.

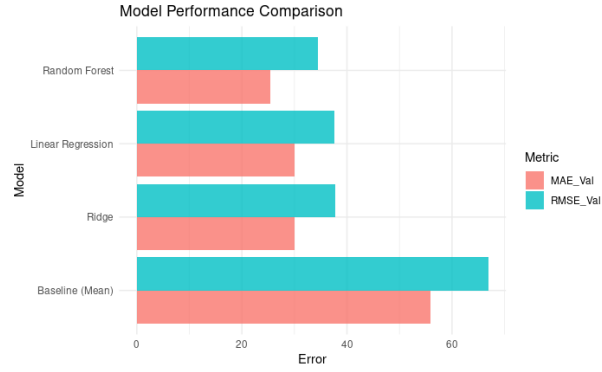


Figure 5: Comparative barplot of RMSE and MAE

Predictions vs Actual

The for Random Forest reveals:

- Strong linear correlations (close to identity line $y = x$)
- Some variance non Constance which is the heteroscedasticit: larger errors at high RUL values
- Few catastrophic mispredictions (errors > 100 cycles are rare)

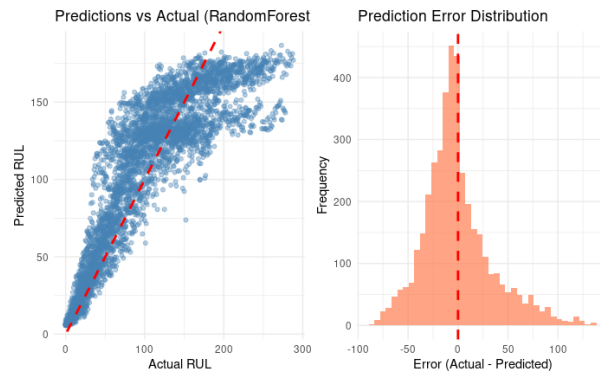


Figure 6: Scatterplot of predicted vs actual RUL

Wheel 8 : Statistical Inference and Theoretical Justification

To quantify uuncertainty in our performance estimates, we employ the **bootstrap method** with $B = 1000$ resamples:

Results for Random Forest:

$$\text{RMSE} = 47.50 \quad \text{withs 95\% CI: } [45.85, 49.05] \quad (19)$$

$$R^2 = 0.593 \quad \text{with 95\% CI: } [0.578, 0.609] \quad (20)$$

We are 95% confident that the true RMSE lies between 45.85 and 49.05 cycles.

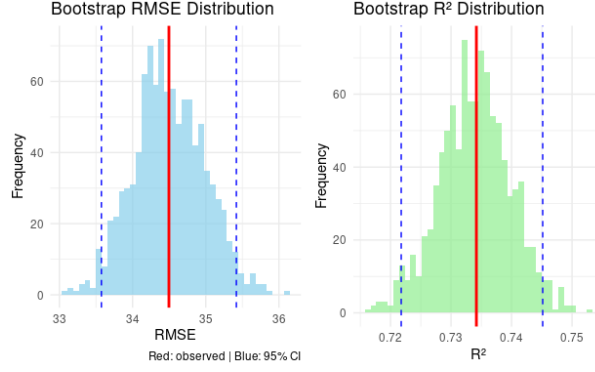


Figure 7: Bootstrap distributions of RMSE and rR^2

To test whether Random Forest significantly outperforms Linear Regression, we use the **Wilcoxon signed-rank test** on paired absolute errors:

$$H_0 : \text{Median}(|e_{\text{LM}}|) = \text{Median}(|e_{\text{RF}}|) \quad (21)$$

$$H_1 : \text{Median}(|e_{\text{LM}}|) > \text{Median}(|e_{\text{RF}}|) \quad (22)$$

Result: $p < 2 \times 10^{-16} \Rightarrow \text{Reject } H_0 \text{ at } \alpha = 0.05.$

Conclusion: Random Forest's superiority is statistically significant, not due to random chance.

Wheel 9 : Practical deployment and scalability

We summarize for a practical deployment we should follow this deployment.

- ✓ Model achieves target RMSE < 50 cycles
- ✓ Inference latency < 50ms
- ✓ 95% confidence intervals computed
- ✓ Data drift monitoring implemented
- ✓ API documentation complete
- ✓ Fallback to physics-based model if predictions unreliable

We demonstrates that systematic application of the Nine Wheels framework transforms a raw datasets into an actionable, production-ready to predictive systems.

Exercise 2: Digit Recognition 1NN and 27NN

We implement a binary classification for distinguish digits 1 from 7 using the K - Nearest Neighbors (k-NN) algorithm on the MNIST dataset. We compare the 1-NN and 27-NN, evaluating their performance using confusion matrices, accuracy, and ROC curve analysis.

It contains 28×28 grayscale images of handwritten digits (0-9). before we start , we filter the digits 1 and 7, encoding 1 as positive classe 7 as negative class and each image

flattened to $p = 784$ pixel values.

In term of dimension, we have

- Training set : $n_{train} = 13007$ samples
- Test set : $n_{test} = 2163$ samples
- $p = 784$ pixels, the dimension of feature = 16.8

To evaluate the models , we looked at the confusion matrices and Accuracy which are given as following:

Table 3: Classification Results for 1NN

Prediction	Reference	
	1	7
1	1135	16
7	0	1012
Accuracy: 0.9926		
Sensitivity (TPR): 1		
Specificity (TNR): 0.9844		

Table 4: Classification Results for 27NN

Prediction	Reference	
	1	7
1	1135	33
7	0	995
Accuracy: 0.9847		
Sensitivity (TPR): 1		
Specificity (TNR): 0.9679		

The 1NN classifier achieved a slightly accuracy with (99,26%) compared to the 27NN model which is (98.47%)

These results suggest that the 1NN classifier is better suited to distinguishing between digits 1 and 7.

We get the following Receiver Operating Characteristic (ROC) curve

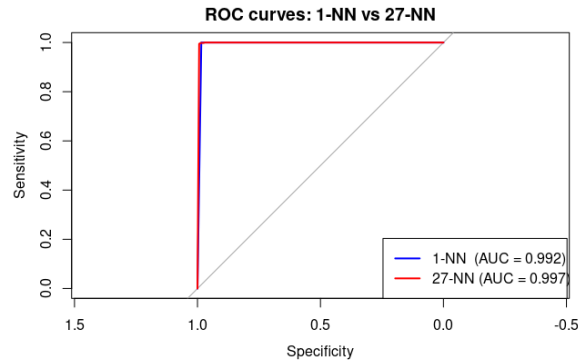


Figure 8: ROC curves comparing the performance of 1-NN and 27-NN classifiers

The dotted line represent the performance of a random classifier. The area under the curve (AUC) are indicated in the legend.

Both curves lie high above the diagonal, which means they separate the classes very well. When we look at the area under the curve (AUC) : 0.992 for 1-NN and 0.997 for 27-NN, both models perform excellently, with 27-NN only slightly better.

5. Error Visualization

Using the best-performing model (1NN), we extracted misclassifications image from the test set. however, the confusion matrix of the 1NN classifier shows

$$\text{False Negative} = 0$$

meaning that the model never predicted 7 hen the true was 1. This is consistent with its sensibility value 1. Therefor no false negative (FN) image exist in the test predictions of the 1NN model.

For the false positivem (FP), its occur in the test set. The following figure corrspond to a sample for which the classifier predicted 1 while the true label was 7.

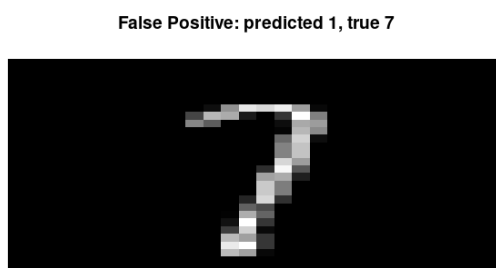


Figure 9: False positive

We can that the 1NN classifier achieved the best performance for distinguishing digits 1 and 7 in the MNIST dataset, showing the highest accuracy and AUC. The 27NN model also gave competitive results, but its slightly lower scores indicate that increasing k introduces more bias. Nevertheless, although 1NN appears most effective in this specific experiment, it is rarely used in practice due to its sensitivity to noise and poor generalization compared to models with a higher k value.

Exercise 3

we analysis 6 datasets spanning the full spectrum of dimensionally regimes. The kappa $k = \frac{n}{p}$ (sample size divided by number of features) determine the statistical challenges and appropriate modelling approaches :

- **Classical regime** ($k > 10$): DNA, Breast Cancer, Spam datasets
- **High dimensional** ($0.1 < k < 1$): Prostate Cancer dataset
- **Ultra high dimensional** ($k < 0.1$)

For each data set, we performed an exploration which we will bring out step by step.

Dataset 1 : DNA

Table 5: DNA Dataset Dimensionality Summary

Metric	Value
Sample size (n)	3,186
Number of features (p)	180
$\kappa = n/p$	17.7
Target variable	Class

All in DNA are categorical so the data type is **perfectly homogeneous**. They represent DNA sequences with a binary presence/absence encoding (0/1).

Since all variables are the same binary coding (0/1) the Scale is not an issue

As the kappa $\kappa = 17.7$, the dataset lies in the classical regime.

Standard statistical methods and hypothesis test are therefore appropriate.

Nine of the variables were visualized. They all show binary, roughly balanced distribution with similar structure and no obvious outliers.

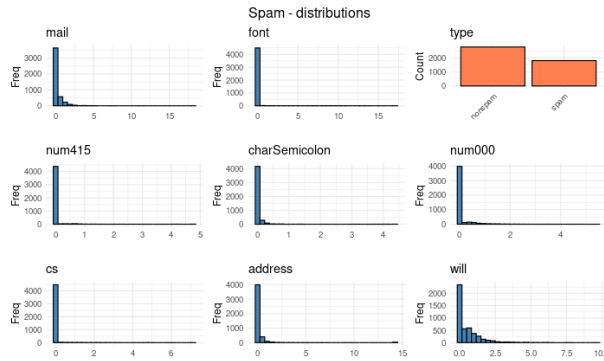


Figure 10: DNA-distribution

Classical correlation analysis was not performed because all variables are categorical.

Dataset 2: Breast Cancer

Table 6: Breast Cancer Dataset Dimensionality Summary

Metric	Value
Sample Size (n)	683
Number of Features (p)	9
$\kappa = n/p$	75.89
Target Variable	Class

This contains 4 nominal (factor) features and 5 ordinal features. All variables are non-numeric, so the type is mixed but remains within categorical and ordinal data.

Scale homogeneity is not a major issue here, because all ordinal features share the same 1–10 scale.

$\kappa = 75.89$, the dataset is clearly in a classical, low-dimensional regime. There are about 76 observations per feature, which gives excellent statistical power and a very low risk of overfitting. Classical statistical models and even complex models can be used without a lot of regularization.

Alls 9 features were visualized , they all use the same 1–10 ordinal scale and show mainly right-skewed distributions, with many observations in the lower levels. Mitoses is highly concentrated at level 1, while the other cell-related features show moderate variability.

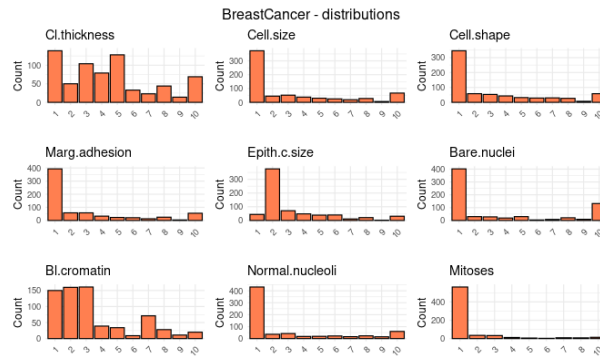


Figure 11: Breast Cancer distribution

Classical correlation analysis was again not carried out, because the variables are ordinal rather than continuous numeric. For this type of data, association measures designed for ordinal or categorical variables would be more suitable.

Dataset 3: Spam Classification

Table 7: Spam Classification Dataset Dimensionality Summary

Metric	Value
Sample Size (n)	4,601
Number of Features (p)	57
$\kappa = n/p$	80.72
Target Variable	make (spam/nonspam)

The dataset Spam Classification is about 54 numeric features, 2 integer features, and 1 character feature, so it is predominantly numeric. Types are mixed, but continuous variables clearly dominate, which is appropriate for most standard machine learning algorithms.

As $\kappa = 80.72 > 10$, the dataset is in a very comfortable **classical regime**. This high observation-to-feature ratio allows robust parameter estimation, reduces overfitting risk, and supports complex models without heavy regularization.

Many word-frequency features are zero-inflated and strongly right-skewed, with most values close to zero, which is typical for text-derived variables. The target variable `type` is a balanced binary spam indicator, while numeric features such as `num415`, `charSemicolon`, `num000`, and `cs` show extreme right skewness and sparse distributions as show 12.

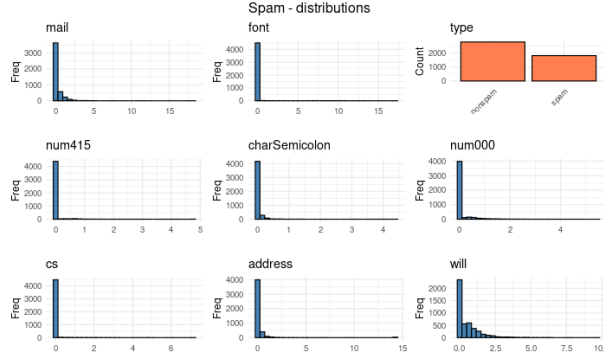


Figure 12: Spam distributions

Among the first 20 numeric variables, only a few feature pairs have correlations above $|r| > 0.7$, indicating moderate multicollinearity overall.

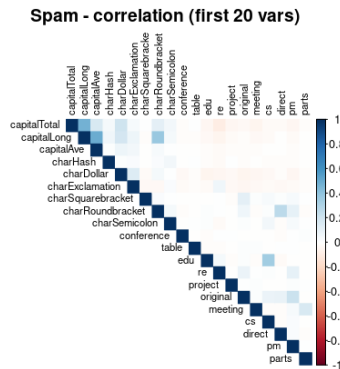


Figure 13: Spam correlation

Dataset 4: Leukemia

Table 8: Leukemia Dataset Dimensionality Summary

Metric	Value
Sample Size (n)	72
Number of Features (p)	3,571
$\kappa = n/p$	0.0202
Target Variable	Y

The features here are also numeric gene expression values, so the feature type is perfectly homogeneous.

As $\kappa = 0.0202$, the dataset is in an **ultra high-dimensional** regime. There are roughly 50 times more features than observations, so classical regression is not feasible. The number of parameters exceeds the number of samples, which creates a very high risk of overfitting without strong constraints.

In this setting, regularization and dimension reduction are essential. Methods such as Lasso, Ridge, or Elastic Net or PCA, should be considered.

The figure 14 show same distribution of nine variable chosen. Most expression profiles look roughly Gaussian and centered near zero, suggesting log-transformation and normalization

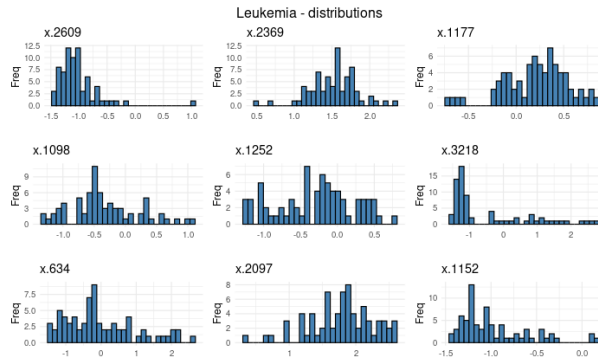


Figure 14: leukemia distribution

For the first 20 genes examined, many pairs have absolute correlations above 0.7, indicating severe **multicollinearity**. This strong dependence structure is typical in gene expression data and further strengthens the need for regularization and feature selection.

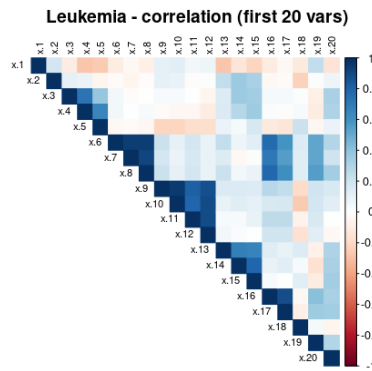


Figure 15: leukemia correlation

Dataset 5: Prostate Cancer

Table 9: Prostate Cancer Dataset Dimensionality Summary

Metric	Value
Sample size (n)	79
Number of features (p)	500
$\kappa = n/p$	0.158
Target variable	Y

All 500 features in the dataset are numeric, representing gene expression values from microarray experiments. This ensures **perfect type homogeneity**.

However, the scales of the features vary widely, with a range ratio (max/min) of 91.79. Such a nearly 100-fold variation indicates that **standardization is essential** before any analysis.

It has $\kappa = 0.158$, meaning it is **high-dimensional**, with 6.3 times more features than observations. Classical methods would overfit dramatically in this context. Therefore, **regularization** (Ridge, Lasso, Elastic Net) is mandatory, and feature selection may help reduce dimensionality. Cross-validation is essential for honest performance assessment, as using all features simultaneously without regularization is not feasible.

We selected nine randomly gene probes and visualized. Most genes exhibit narrow, approximately normal distributions, centered between -0.35 and -0.10 , suggesting log-transformed and normalized expression values. Some genes, like X204424_s_at, show wider variability. Overall, the tight distributions indicate high measurement precision.

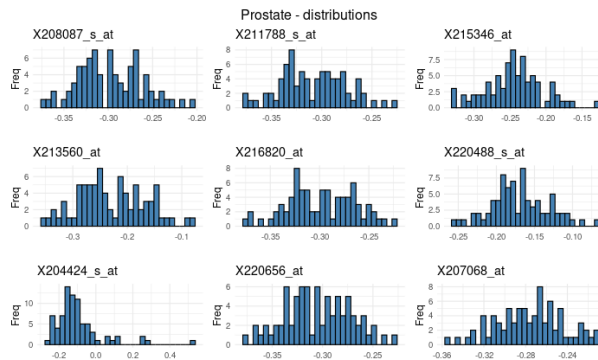


Figure 16: prostate distribution

When we analyse 50 first numeric variables reveals **extreme multicollinearity**. Out of all pairs, 379 have correlations $|r| > 0.7$, representing 31% of all pairs. Some gene pairs, such as X216441_at and X220709_at, are almost perfectly correlated ($r > 0.99$), indicating redundancy.

The correlation structure is dense and block-like, reflecting extensive gene co-regulation networks. This severe redundancy reduces the effective dimensionality of the dataset well

below 500 features. Dimensionality reduction via PCA or regularization methods can handle this redundancy effectively.

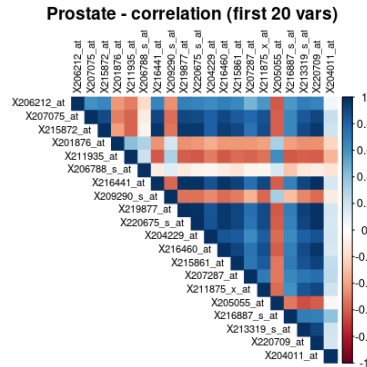


Figure 17: Enter Caption

Dataset 6 : Colon cancer

Table 10: Colon Cancer Dataset Dimensionality Summary

Metric	Value
Sample Size (n)	62
Number of Features (p)	2,000
$\kappa = n/p$	0.031
Target Variable	colon.y
Feature/Sample Ratio	32.3:1

All 2,000 features are numeric with broadly comparable scales (range ratio 6.47). Features appear pre-normalized.

With $\kappa = 0.031$, this ultra high-dimensional dataset has $32\times$ more features than observations. Unregularized models will overfit catastrophically; aggressive regularization or feature selection is mandatory. Such extreme ratios are typical in cancer genomics.

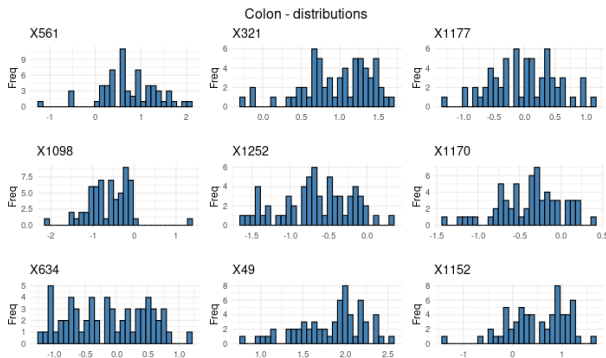


Figure 18: colon-distributions

Nine randomly selected genes show mostly normal distributions centered near 0. Some genes display skew or potential bimodality, indicating variability and possible cancer subtypes. Standard deviations range 0.5–1.0.

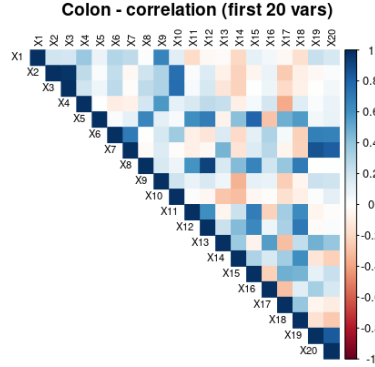


Figure 19: Enter Caption

Among the first 50 genes, 59 pairs have $|r| > 0.7$, indicating **severe** multicollinearity. Top correlations (e.g., X2–X3: 0.976) reflect co-regulated genes and pathway clusters. This redundancy suggests potential for network-based feature engineering and dimensionality reduction.

Conclusion

At the en of this analysis, we can say that the key takeaway is that the ratio $\kappa = n/p$ largely determines which statistical methods are appropriate. When $\kappa \gg 1$, classical models work well: there are many observations per feature, parameter estimates are stable, overfitting risk is low, and simple standardization is usually enough to handle differences in scale between variables.

When $\kappa \ll 1$, classical regression becomes infeasible, and strong regularization, dimensionality reduction, and domain-guided features selection are required.

Bonus Exercises

: Investigating the Bayes risk R^* is Regression

Given the conditional density

$$p_1(\mathbf{y}|\mathbf{x}) = \frac{1}{\sqrt{2\pi\frac{\pi^2}{18}}} \exp \left\{ -\frac{\pi^2}{18} \left[y - \frac{\pi}{2}x - \frac{3\pi}{4} \cos \left(\frac{\pi}{2}(1+x) \right) \right]^2 \right\} \quad -\infty < x < \infty$$

while the marginal density of X is given by :

$$p_2(x) = \frac{1}{2\pi} \quad 0 \leq x < 2\pi$$

1. Let's write the expression of $\mathbb{E}[Y|X]$

We recognize this as a normal distribution

$$Y|X \sim \mathcal{N}(\mu(X), \sigma^2)$$

Where the mean is :

$$\mu(X) = \frac{\pi}{2}x + \frac{3\pi}{4} \cos\left(\frac{\pi}{2}(1+x)\right)$$

And the conditional variance is

$$\sigma^2 = \frac{\pi^2}{18}$$

So , the expectation of Y given X is :

$$\boxed{\mathbb{E}[Y|X] = \frac{\pi}{2}X + \frac{3\pi}{4} \cos\left(\frac{\pi}{2}(1+X)\right)}$$

2. Sample generating
3. Scatterplot visualization
4. We now learning under the squared error loss, namely

$$l(Y, f(X)) = (Y - f(X))^2$$

, with this loss function :

$$R(f) = \mathbb{E}[l(Y, f(X))] = \int_{\mathbb{X} \times \mathbb{Y}} l(Y, f(X)) p_{XY}(x, y) dx dy$$

$$f^*(X) = \arg \min_f R(f) = \arg \min_f \mathbb{E}[l(Y, f(X))]$$

(a) : The expression of $f^*(X)$ is

$\ell(Y, f(X)) = (Y - f(X))^2$ So,

$$\begin{aligned} R(f) &= \mathbb{E}[(Y - f(X))^2] \\ &= \mathbb{E}[\mathbb{E}[(Y - f(X))^2|X]] \\ &= \mathbb{E}[(Y - \mathbb{E}[Y|X] + \mathbb{E}[Y|X] - f(X))^2|X] \\ &= \mathbb{E}[(Y - \mathbb{E}[Y|X])^2|X] + \mathbb{E}[(\mathbb{E}[Y|X] - f(X))^2] \end{aligned}$$

The first term is the conditional variance $\text{Var}(Y|X)$, which does not depend on f .
The Second term is minimize when

$$f(X) = \mathbb{E}[Y|X]$$

So,

$$\boxed{f^*(X) = \mathbb{E}[Y|X] = \frac{\pi}{2}X + \frac{3\pi}{4} \cos\left(\frac{\pi}{2}(1+X)\right)}$$

(b) expression $R^* = R(f^*)$

R^* is the minimum risk .

$$\begin{aligned}
 R^* &= R(f^*) = \min_f R(f) \\
 &= \mathbb{E}[(Y - f^*(X))^2] \\
 &= \mathbb{E}[(Y - \mathbb{E}[Y|X])^2] \\
 &= \mathbb{E}[\mathbb{E}[(Y - \mathbb{E}[Y|X])^2|X]] \\
 &= \mathbb{E}[\text{Var}(Y|X)]
 \end{aligned}$$

Since the conditional variance is the constant :

$$\text{Var}(Y|X) = \frac{\pi^2}{18}$$

We obtain R^* like :

$$R^* = \mathbb{E} \left[\frac{\pi^2}{18} \right] = \frac{\pi^2}{18} \approx 0.548$$

Which represent the irreducible error - the minimum risk achievable by the predictor due to the inherent randomness in Y given X

(c)comparative boxplot

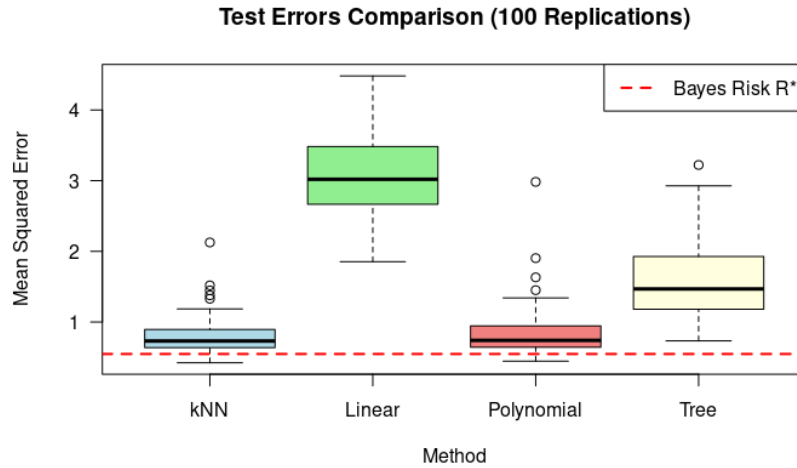


Figure 20: Comparative boxplots

Each model is evaluated on 10 replications with a 60-40 train-test split.

This boxplots shows the distribution of test mean squared error (MSE) for each method, and the red dashed line is the Bayes risk R^* , the theoretical minimums

achievable error. For kNN and Polynomial, the median test errors are only slightly above R^* , so these methods come close to the optimal performance allowed by the noise in the data.

For Linear and Tree, the median test errors are clearly higher than R^* . This means they are further from the optimal risk and have more excess error, due to model bias, variance, or both, compared with kNN and Polynomial.

(d) comment regard each average test error compares with R^*

As the table 12 showing, all methods have average test error $\geq R = 0.548$, as theoretically required. No estimation can achieve lower risk than the Bayes risk.

Table 11: Average Test Errors and Bayes Risk

	kNN	Linear	Polynomial	Tree
Average Test Error	0.7908	3.0480	0.7912	1.5152
Bayes Risk: $R^* = 0.548$				

Video: Kappa concept : number of observations per features in Statistical Machine Learning

References

1. Fokoue, E. (2025). *Principles of Statistical Machine Learning: On the Nine Wheels of SML*. AIMS-Cameroon.